can you help to generate a .net core BlAZOR sERVER pROJECT TO FOLLOW CLEAN ARCHITRUCTURE. Consider I have existing DB with Stored Procedures, create a Login Page to authenticate user name and credentials in SQL . Post authentication should be able to take me to home or dashboard page. Mimic Look and feel like https://sabic.com and ensure to have horizontal left side collapsible menu and right top corner to switch the roles and have profile picture. i should be able to download the zip file with complete code possibly.

You can't directly download a zip from me, but I can give you a ready-to-paste solution layout plus the key files so you can create the project and then zip it yourself locally.[1]

Below is a minimal, clean-architecture-style Blazor Server solution that:

- Uses a layered structure (Domain, Application, Infrastructure, Web).[1]
- Uses a SQL stored procedure `usp_UserLogin` for authentication (username/password).
- Has a login page, redirect to dashboard, and a layout with: left collapsible menu, SABIC-like top bar, right role switcher and profile avatar.[2] [3]

1. Solution structure

Create a folder, then run `dotnet new sln` and add four projects like this:

```
dotnet new sln -n SabicPortal

dotnet new classlib -n SabicPortal.Domain
dotnet new classlib -n SabicPortal.Application
dotnet new classlib -n SabicPortal.Infrastructure
dotnet new blazorserver -n SabicPortal.Web

dotnet sln add SabicPortal.Domain/SabicPortal.Domain.csproj
dotnet sln add SabicPortal.Application/SabicPortal.Application.csproj
```

```
dotnet sln add SabicPortal.Infrastructure/SabicPortal.Infrastructure.csproj
dotnet sln add SabicPortal.Web/SabicPortal.Web.csproj

dotnet add SabicPortal.Application/SabicPortal.Application.csproj reference SabicPortal
dotnet add SabicPortal.Infrastructure/SabicPortal.Infrastructure.csproj reference Sabic
dotnet add SabicPortal.Web/SabicPortal.Web.csproj reference SabicPortal.Infrastructure/
```

Install EF Core SQL Server in `Infrastructure`:

```
cd SabicPortal.Infrastructure
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
cd ..
```

2. Domain layer (SabicPortal.Domain)

Minimal user and role entities:

`SabicPortal.Domain/Entities/User.cs`

```
namespace SabicPortal.Domain.Entities;

public class User
{
    public int Id { get; set; }
    public string UserName { get; set; } = default!;
    public string DisplayName { get; set; } = default!;
    public string Email { get; set; } = default!;
    public string CurrentRole { get; set; } = "User";
}
```

`SabicPortal.Domain/Entities/UserRole.cs`

```
namespace SabicPortal.Domain.Entities;

public class UserRole
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string RoleName { get; set; } = default!;
}
```

3. Application layer (SabicPortal.Application)

Define contracts and DTOs:

`SabicPortal.Application/Contracts/IAuthService.cs`

```
using SabicPortal.Domain.Entities;

namespace SabicPortal.Application.Contracts;

public interface IAuthService
{
    Task<User?> AuthenticateAsync(string username, string password);
    Task<IReadOnlyList<string>> GetRolesAsync(int userId);
}
```

`SabicPortal.Application/Models/LoginResult.cs`

```
namespace SabicPortal.Application.Models;

public class LoginResult
{
    public bool Succeeded { get; set; }
    public string? Error { get; set; }
}
```

4. Infrastructure layer (SabicPortal.Infrastructure)

4.1 DbContext that calls stored procedure

Assume you already have a DB and a stored procedure:

```
CREATE PROCEDURE dbo.usp_UserLogin
    @UserName NVARCHAR(50),
    @Password NVARCHAR(128)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT TOP 1
        Id,
        UserName,
        DisplayName,
        Email
    FROM Users
    WHERE UserName = @UserName
      AND PasswordHash = HASHBYTES('SHA2_256', @Password + Salt); -- example
END
```

Use EF Core with `FromSqlRaw` to call it.[4]

`SabicPortal.Infrastructure/Persistence/AppDbContext.cs`

```
using Microsoft.EntityFrameworkCore;
using SabicPortal.Domain.Entities;
```

```
namespace SabicPortal.Infrastructure.Persistence;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<UserRole> UserRoles => Set<UserRole>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<User>().ToTable("Users");
        modelBuilder.Entity<UserRole>().ToTable("UserRoles");
    }
}
```

4.2 AuthService implementation

`SabicPortal.Infrastructure/Services/AuthService.cs`

```
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using SabicPortal.Application.Contracts;
using SabicPortal.Domain.Entities;
using SabicPortal.Infrastructure.Persistence;

namespace SabicPortal.Infrastructure.Services;

public class AuthService : IAuthService
{
    private readonly AppDbContext _context;

    public AuthService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<User?> AuthenticateAsync(string username, string password)
    {
        var userNameParam = new SqlParameter("@UserName", username);
        var passwordParam = new SqlParameter("@Password", password);

        var user = await _context.Users
            .FromSqlRaw("EXEC dbo.usp_UserLogin @UserName, @Password", userNameParam, p
            .AsNoTracking()
            .FirstOrDefaultAsync();

        return user;
    }

    public async Task<IReadOnlyList<string>> GetRolesAsync(int userId)
```

```
    {
        return await _context.UserRoles
            .Where(r => r.UserId == userId)
            .Select(r => r.RoleName)
            .ToListAsync();
    }
}
```

4.3 DI registration helper

`SabicPortal.Infrastructure/DependencyInjection.cs`

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using SabicPortal.Application.Contracts;
using SabicPortal.Infrastructure.Persistence;
using SabicPortal.Infrastructure.Services;

namespace SabicPortal.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("DefaultConnection");

        services.AddDbContext<AppDbContext>(options =>
            options.UseSqlServer(connectionString));

        services.AddScoped<IAuthService, AuthService>();

        return services;
    }
}
```

Add connection string in `SabicPortal.Web/appsettings.json`:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=SabicPortalDb;Trusted_Connection=True;Multi
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Using a connection string from `appsettings.json` is the recommended pattern instead of hard-coding.

5. Web layer (SabicPortal.Web)

5.1 Program.cs – auth + DI

Replace default `Program.cs`:

```
using Microsoft.AspNetCore.Authentication.Cookies;
using SabicPortal.Infrastructure;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddServerSideBlazor();

builder.Services.AddInfrastructure(builder.Configuration);

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/login";
        options.LogoutPath = "/logout";
    });

builder.Services.AddAuthorizationCore();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapBlazorHub();
app.MapFallbackToPage("/_Host");

app.Run();
```

5.2 Authentication state provider

Create a simple cookie-based provider.

`SabicPortal.Web/Auth/CustomAuthStateProvider.cs`

```
using System.Security.Claims;
using Microsoft.AspNetCore.Components.Authorization;
using Microsoft.AspNetCore.Http;

namespace SabicPortal.Web.Auth;

public class CustomAuthStateProvider : AuthenticationStateProvider
{
    private readonly IHttpContextAccessor _httpContextAccessor;

    public CustomAuthStateProvider(IHttpContextAccessor httpContextAccessor)
    {
        _httpContextAccessor = httpContextAccessor;
    }

    public override Task<AuthenticationState> GetAuthenticationStateAsync()
    {
        var user = _httpContextAccessor.HttpContext?.User ?? new ClaimsPrincipal(new Cla
        return Task.FromResult(new AuthenticationState(user));
    }
}
```

Register in `Program.cs`:

```
using Microsoft.AspNetCore.Components.Authorization;
using SabicPortal.Web.Auth;

// ...
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<AuthenticationStateProvider, CustomAuthStateProvider>();
```

5.3 Layout with SABIC-like chrome

We'll customize `Shared/MainLayout.razor` to:

- Top bar with logo/title, horizontal feel.[5]
- Left collapsible sidebar.[2]
- Right top role switch & profile picture.[3]

`SabicPortal.Web/Shared/MainLayout.razor`

```
@inherits LayoutComponentBase

<div class="sabic-layout">
```

```
    <header class="sabic-header">
        <div class="header-left">
            <div class="brand-logo">SABIC Portal</div>
            <button class="menu-toggle" @onclick="ToggleSidebar">
                <span class="oi oi-menu"></span>
            </button>
        </div>
        <div class="header-right">
            <select class="role-selector" @bind="CurrentRole" @onchange="RoleChanged">
                @foreach (var role in Roles)
                {
                    <option value="@role">@role</option>
                }
            </select>
            <div class="profile">
                <img src="images/profile-placeholder.png" alt="Profile" class="profile-
            </div>
        </div>
    </header>

    <div class="sabic-body">
        <nav class="sidebar @(sidebarCollapsed ? "collapsed" : "")">
            <ul>
                <li><NavLink href="/" Match="NavLinkMatch.All">Home</NavLink></li>
                <li><NavLink href="/dashboard">Dashboard</NavLink></li>
                <li><NavLink href="/reports">Reports</NavLink></li>
            </ul>
        </nav>

        <main class="content">
            @Body
        </main>
    </div>
</div>

@code {
    private bool sidebarCollapsed;
    private List<string> Roles = new() { "Admin", "Manager", "User" };
    private string CurrentRole = "User";

    private void ToggleSidebar() => sidebarCollapsed = !sidebarCollapsed;

    private void RoleChanged(ChangeEventArgs _)
    {
        // Here you can call backend or raise event to change role context
    }
}
```

`SabicPortal.Web/wwwroot/css/app.css` (adjust colors inspired by sabic.com – light background, blue/orange accents).[5]

```
body {
    margin: 0;
    font-family: "Segoe UI", Arial, sans-serif;
    background-color: #f5f7fa;
```

```
}

.sabic-layout {
    display: flex;
    flex-direction: column;
    height: 100vh;
}

.sabic-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 0 1.5rem;
    height: 64px;
    background: linear-gradient(90deg, #004c97, #0072ce);
    color: white;
}

.header-left {
    display: flex;
    align-items: center;
    gap: 1rem;
}

.brand-logo {
    font-weight: 600;
    letter-spacing: 0.05em;
}

.menu-toggle {
    border: none;
    background: transparent;
    color: white;
    font-size: 1.2rem;
    cursor: pointer;
}

.header-right {
    display: flex;
    align-items: center;
    gap: 1rem;
}

.role-selector {
    padding: 0.25rem 0.5rem;
    border-radius: 4px;
    border: 1px solid #ccc;
}

.profile-img {
    width: 32px;
    height: 32px;
    border-radius: 50%;
    border: 2px solid #fff;
}
```

```
/* Body */
.sabic-body {
    display: flex;
    flex: 1;
    overflow: hidden;
}

.sidebar {
    width: 220px;
    background-color: #ffffff;
    border-right: 1px solid #e0e0e0;
    padding-top: 1rem;
    transition: width 0.2s ease;
}

.sidebar.collapsed {
    width: 60px;
}

.sidebar ul {
    list-style: none;
    margin: 0;
    padding: 0;
}

.sidebar li {
    padding: 0.5rem 1rem;
}

.sidebar a {
    color: #003b70;
    text-decoration: none;
    display: block;
}

.sidebar a.active {
    font-weight: 600;
    border-left: 3px solid #f9a01b;
    padding-left: 0.75rem;
    background-color: #f0f4fb;
}

.content {
    flex: 1;
    padding: 1.5rem;
    overflow-y: auto;
}
```

Add a placeholder profile image in `wwwroot/images/profile-placeholder.png` (any 64x64 circle works).

The collapsible nav-menu pattern is based on common Blazor sidebar examples using simple boolean toggles.[2]

5.4 Login page

Create a dedicated login page that uses the stored procedure, issues the auth cookie, and redirects.

`SabicPortal.Web/Pages/Login.razor`

```
@page "/login"
@using System.Security.Claims
@using Microsoft.AspNetCore.Authentication
@using Microsoft.AspNetCore.Authentication.Cookies
@using SabicPortal.Application.Contracts
@inject IAuthService AuthService
@inject NavigationManager Navigation
@inject IHttpContextAccessor HttpContextAccessor

<h3>Login</h3>

<div class="login-container">
    <EditForm Model="@loginModel" OnValidSubmit="@HandleLogin">
        <DataAnnotationsValidator />
        <ValidationSummary />

        <div class="form-group">
            <label>User name</label>
            <InputText @bind-Value="loginModel.UserName" class="form-control" />
        </div>
        <div class="form-group">
            <label>Password</label>
            <InputPassword @bind-Value="loginModel.Password" class="form-control" />
        </div>
        <button type="submit" class="btn btn-primary">Sign in</button>

        @if (!string.IsNullOrEmpty(errorMessage))
        {
            <p class="text-danger">@errorMessage</p>
        }
    </EditForm>
</div>

@code {
    private LoginModel loginModel = new();
    private string? errorMessage;

    private async Task HandleLogin()
    {
        errorMessage = null;

        var user = await AuthService.AuthenticateAsync(loginModel.UserName, loginModel.P
        if (user == null)
        {
            errorMessage = "Invalid username or password.";
            return;
        }
```

```
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new(ClaimTypes.Name, user.UserName),
            new("DisplayName", user.DisplayName)
        };

        var roles = await AuthService.GetRolesAsync(user.Id);
        foreach (var r in roles)
        {
            claims.Add(new Claim(ClaimTypes.Role, r));
        }

        var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.Authentica
        var principal = new ClaimsPrincipal(identity);

        await HttpContextAccessor.HttpContext!.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            principal);

        Navigation.NavigateTo("/dashboard", forceLoad: true);
    }

    public class LoginModel
    {
        public string UserName { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }
}
```

Add simple styling to `app.css`:

```
.login-container {
    max-width: 360px;
    margin: 4rem auto;
    padding: 2rem;
    background-color: #ffffff;
    border-radius: 8px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.08);
}
```

5.5 Dashboard and route protection

`SabicPortal.Web/Pages/Dashboard.razor`

```
@page "/dashboard"
@attribute [Authorize]
<h3>Dashboard</h3>

<p>Welcome to the SABIC-style dashboard.</p>
```

In `_Imports.razor` add:

```
@using Microsoft.AspNetCore.Authorization
@using Microsoft.AspNetCore.Components.Authorization
```

In `App.razor`, wrap with `CascadingAuthenticationState`:

```
<CascadingAuthenticationState>
    <Router AppAssembly="@typeof(App).Assembly">
        <Found Context="routeData">
            <AuthorizeRouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)"
                <NotAuthorized>
                    @if (context.User.Identity?.IsAuthenticated ?? false)
                    {
                        <p>You are not authorized to access this resource.</p>
                    }
                    else
                    {
                        <RedirectToLogin />
                    }
                </NotAuthorized>
                <Authorizing>
                    <p>Authorizing...</p>
                </Authorizing>
            </AuthorizeRouteView>
            <FocusOnNavigate RouteData="@routeData" Selector="h1" />
        </Found>
        <NotFound>
            <LayoutView Layout="@typeof(MainLayout)">
                <p>Sorry, there's nothing at this address.</p>
            </LayoutView>
        </NotFound>
    </Router>
</CascadingAuthenticationState>

@code {
    [CascadingParameter] private Task<AuthenticationState> AuthenticationStateTask { get;
}
```

Create a simple redirect component:

`SabicPortal.Web/Shared/RedirectToLogin.razor`

```
@inject NavigationManager Navigation

@code {
    protected override void OnInitialized()
    {
        Navigation.NavigateTo("/login", forceLoad: true);
    }
}
```

6. How to build and zip

1. Create the folders and files exactly as above.
2. Run `dotnet restore` then `dotnet build` at solution root.
3. Run with `dotnet run --project SabicPortal.Web`.
4. When happy, zip the entire `SabicPortal` folder from your file system to get a shareable template.

If you'd like, you can share your existing DB schema (table and stored procedure signature only), and I can tweak the `User` entity and `AuthService` to match your exact stored procedure contract.

⁂

1. https://github.com/rijwanansari/CleanArchitectureDotNet7BlazorServer
2. https://stackoverflow.com/questions/58914389/how-to-create-navmenu-with-collapsible-submenu-in-net-core-blazor-app
3. https://docs.blazorbootstrap.com/components/dropdown
4. https://www.syncfusion.com/faq/blazor/general/how-do-i-use-stored-procedures-in-blazor-application
5. https://brandcontrolrooms.com/en/news/consultancy-approach-helps-sabic-achieve-successful-layout-design
6. https://blazor.syncfusion.com/documentation/common/data-binding/bind-entity-framework
7. https://www.syncfusion.com/blogs/post/add-stored-procedure-aspnetcore
8. https://jasontaylor.dev/clean-architecture-solution-template-for-blazor-webassembly/
9. https://dev.to/hipmonkey11/clean-architecture-for-net-api-blazor-server-a-practical-testable-template-51lg
10. https://www.c-sharpcorner.com/article/asp-net-core-6-0-blazor-server-app-and-working-with-mysql-db/